Spring Boot Fundamentals ��� Interview Guide

Spring Boot Fundamentals — Interview-Ready Master Guide

> *Covers: notes, internal working, real-world examples, complete code, 30–40 interview Q&A, coding problems, AI-style questions, and a mock interview transcript.*

PART 1: CORE CONCEPTS — NOTES

1.1 Spring vs Spring Boot

Spring Framework is a large ecosystem for building Java applications, built around two central ideas: **Dependency Injection (DI)** and **Inversion of Control (IoC)**. It gives you modules for almost everything (Spring MVC for web, Spring Data for persistence, Spring Security, etc.), but you wire and configure most of it yourself — XML config, `@Configuration` classes, manually declared beans, manually added servlet containers.

Spring Boot is built *on top of* Spring. It doesn't replace Spring — it's an opinionated, convention-over-configuration layer whose entire purpose is to eliminate the boilerplate of setting Spring up. Its main contributions:

	Spring	Spring Boot
Configuration	Mostly manual (XML or Java config)	Auto-configuration based on classpath contents
Server	You deploy a WAR to an external Tomcat/Jetty	Embedded server (Tomcat by default) — runs as a plain `java -jar`
Dependency management	You pick and align every version yourself	"Starter" dependencies bundle compatible versions
Boilerplate	`web.xml`, `DispatcherServlet` setup, etc.	`@SpringBootApplication` + `main()` method, that's it
Philosophy	Flexible, unopinionated	Opinionated defaults, override only what you need

In short: **Spring is the engine; Spring Boot is the car** — pre-assembled, with sensible defaults, so you can drive immediately instead of bolting the engine together yourself.

1.2 Dependency Injection (DI)

DI is a design pattern where an object's dependencies are **provided to it from the outside**, rather than the object creating them itself.

```
// Without DI — tightly coupled, hard to test
public class OrderService {
    private PaymentGateway gateway = new StripeGateway(); // hardcoded
}

// With DI — the dependency is handed in
public class OrderService {
    private final PaymentGateway gateway;

    public OrderService(PaymentGateway gateway) {   // constructor injection
        this.gateway = gateway;
    }
}
```

Three injection styles in Spring: 1. **Constructor injection** (recommended) — dependencies passed via constructor; enables `final` fields, immutability, and easy testing (just call `new OrderService(mockGateway)` in a unit test, no Spring container needed). 2. **Setter injection** — dependencies set via setter methods; useful for optional dependencies. 3. **Field injection** (`@Autowired` directly on a field) — most concise, but discouraged: it hides dependencies, can't be `final`, and makes plain unit testing harder without reflection tricks.

1.3 Inversion of Control (IoC)

IoC is the *principle*; DI is one common *implementation* of it. Normally, your code controls the flow — it decides when to create objects and call methods ("I create a `PaymentGateway`, then I use it"). With IoC, that control is inverted: a **container** (the Spring `ApplicationContext`) creates, configures, and wires your objects for you, and hands you the finished, ready-to-use object graph. You stop calling `new` for anything managed by Spring; the framework calls into *your* code, not the other way around — hence "inversion."

1.4 Bean

A **bean** is simply an object that the Spring IoC container instantiates, configures, wires together, and manages the lifecycle of. Ways to declare one:

```
// 1. Component scanning — annotate the class itself
@Service
public class OrderService { ... }

// 2. Explicit declaration in a @Configuration class
@Configuration
public class AppConfig {
    @Bean
    public PaymentGateway paymentGateway() {
        return new StripeGateway();
    }
}
```

`@Component` (and its specializations `@Service`, `@Repository`, `@Controller` / `@RestController`) mark a class for **component scanning** — Spring finds it automatically by scanning the classpath. `@Bean` is used inside a `@Configuration` class for cases where you don't own the class's source (e.g., a third-party library object) or need custom construction logic.

Bean scopes: - `singleton` (default) — one shared instance per Spring container. - `prototype` — a new instance every time it's requested. - `request`, `session`, `application` — web-aware scopes, tied to an HTTP request/session/context lifetime.

1.5 Maven

Maven is the (most common) build tool and dependency manager for Java/Spring Boot projects, driven by a `pom.xml` file.

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.0</version>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>1.0.0</version>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Key ideas: the `spring-boot-starter-parent` parent POM manages consistent dependency **versions** for you (no more version-mismatch hell); `spring-boot-starter-*` artifacts are curated bundles (e.g., `spring-boot-starter-web` pulls in Spring MVC + embedded Tomcat + Jackson, all pinned to compatible versions); the `spring-boot-maven-plugin` is what makes `mvn package` produce an executable, self-contained “fat JAR.”

1.6 application.properties

The central place for externalized configuration, loaded automatically from `src/main/resources/`.

```
# Server
server.port=8081

# Datasource
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Logging
logging.level.org.springframework=INFO
logging.level.com.example=DEBUG

# Custom app property
app.jwt.secret=${JWT_SECRET:default-dev-secret}
```

Equivalent YAML (`application.yml`) is also supported and often preferred for nested config readability:

```yaml
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
```

Profiles let you have environment-specific overrides: `application-dev.properties` , `application-prod.properties` , activated via `spring.profiles.active=dev` .

Binding custom properties to a type-safe class:

```java
@ConfigurationProperties(prefix = "app.jwt")
public record JwtProperties(String secret, long expiryMs) {}
```

1.7 Project Structure

A conventional Spring Boot (Maven) layout:

```
demo/
|-- pom.xml
|-- src/
|   |-- main/
|   |   |-- java/
|   |   |   `-- com/example/demo/
|   |   |       |-- DemoApplication.java       # @SpringBootApplication + main()
|   |   |       |-- controller/
|   |   |       |   `-- EmployeeController.java
|   |   |       |-- service/
|   |   |       |   `-- EmployeeService.java
|   |   |       |-- repository/
|   |   |       |   `-- EmployeeRepository.java
|   |   |       |-- model/  (or "entity/")
|   |   |       |   `-- Employee.java
|   |   |       |-- dto/
|   |   |       |   `-- EmployeeDto.java
|   |   |       |-- config/
|   |   |       |   `-- SecurityConfig.java
|   |   |       `-- exception/
|   |   |           `-- GlobalExceptionHandler.java
|   |   `-- resources/
|   |       |-- application.properties
|   |       |-- static/          # served as-is (CSS/JS for a simple front end)
|   |       `-- templates/       # Thymeleaf templates, if used
|   `-- test/
|       `-- java/com/example/demo/
|           `-- EmployeeServiceTest.java
`-- target/                       # build output (compiled classes, the final JAR)
```

This layered structure — controller → service → repository → model — mirrors the typical request flow and is the de facto standard across the Spring Boot ecosystem, which is part of why Spring Boot code across different companies tends to look so similar.

PART 2: CODING — A BASIC SPRING BOOT APPLICATION ([Code])

2.1 Complete, Runnable Example

`pom.xml` (minimal, web + JPA + H2 in-memory DB):

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.0</version>
    </parent>
    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>1.0.0</version>
    <properties><java.version>17</java.version></properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

`DemoApplication.java` — the entry point:

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

`Employee.java` (entity):

```java
package com.example.demo.model;

import jakarta.persistence.*;

@Entity
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String department;

    public Employee() {}
    public Employee(String name, String department) {
        this.name = name;
        this.department = department;
    }

    // getters and setters
    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDepartment() { return department; }
    public void setDepartment(String department) { this.department = department; }
}
```

`EmployeeRepository.java`:

```java
package com.example.demo.repository;

import com.example.demo.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    java.util.List<Employee> findByDepartment(String department);
}
```

`EmployeeService.java`:

```java
package com.example.demo.service;

import com.example.demo.model.Employee;
import com.example.demo.repository.EmployeeRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {   // constructor injection
        this.repository = repository;
    }

    public Employee create(Employee e) { return repository.save(e); }
    public List<Employee> all() { return repository.findAll(); }
    public Employee byId(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Employee not found: " + id));
    }
}
```

`EmployeeController.java`:

```java
package com.example.demo.controller;

import com.example.demo.model.Employee;
import com.example.demo.service.EmployeeService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @PostMapping
    public Employee create(@RequestBody Employee employee) {
        return service.create(employee);
    }

    @GetMapping
    public List<Employee> all() {
        return service.all();
    }

    @GetMapping("/{id}")
    public Employee byId(@PathVariable Long id) {
        return service.byId(id);
    }
}
```

`application.properties`:

```
spring.datasource.url=jdbc:h2:mem:demodb
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
server.port=8080
```

Run it: `mvn spring-boot:run`, then:

```
POST http://localhost:8080/api/employees   {"name":"Alice","department":"Engineering"}
GET  http://localhost:8080/api/employees
GET  http://localhost:8080/api/employees/1
```

That's a complete, working Spring Boot REST API — no `web.xml`, no manually configured servlet container, no manually wired beans. `@SpringBootApplication` and a handful of stereotype annotations are the entire "framework setup."

PART 3: INTERNAL WORKING ([Internal Working])

3.1 What `@SpringBootApplication` Actually Does

It's a meta-annotation — shorthand for three annotations stacked together:

```
        @SpringBootConfiguration   // a specialization of @Configuration — this class can define @Bean methods
        @EnableAutoConfiguration   // triggers Spring Boot's auto-configuration mechanism
        @ComponentScan             // scans this package and sub-packages for
@Component/@Service/@Repository/@Controller
        public @interface SpringBootApplication { ... }
```

This is why placing `DemoApplication` in the root package (`com.example.demo`) matters — `@ComponentScan` defaults to scanning the current package and everything beneath it, so classes outside that tree get silently skipped unless you widen the scan explicitly.

3.2 The Startup Sequence, Step by Step

1. **`SpringApplication.run()`** is called from `main()` . It creates a `SpringApplication` instance and calls `run()` .
2. Spring Boot determines the **application type** (Servlet web, Reactive web, or none) by inspecting the classpath (presence of `spring-webmvc` vs `spring-webflux`).
3. It creates the appropriate **`ApplicationContext`** implementation (e.g., `AnnotationConfigServletWebServerApplicationContext`).
4. **Environment preparation**: `application.properties` / `.yml` , environment variables, command-line args, and active profiles are all merged into a single `Environment` object, in a well-defined precedence order (command-line args generally win over properties files).
5. **`@ComponentScan`** runs, discovering all `@Component` -annotated classes on the classpath under the base package, and registers them as `BeanDefinition` s (metadata about how to build each bean — not the bean instance itself yet).
6. **`@EnableAutoConfiguration`** kicks in: Spring Boot reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (a file bundled inside `spring-boot-autoconfigure.jar`) — a list of hundreds of candidate `@Configuration` classes, each guarded by conditions.
7. Each auto-configuration class is evaluated against **conditional annotations** like `@ConditionalOnClass` (is this library on the classpath?), `@ConditionalOnMissingBean` (has the user already defined their own bean of this type?), `@ConditionalOnProperty` (is a specific property set?). Only the ones whose conditions pass actually register beans. This is the mechanism that makes auto-configuration "smart" — e.g., `DataSourceAutoConfiguration` only activates if a JDBC driver is on the classpath *and* you haven't already defined your own `DataSource` bean.
8. The IoC container instantiates all registered beans, resolving constructor/field dependencies between them (this is where DI actually happens — Spring builds a dependency graph and instantiates beans in an order that satisfies it, detecting circular dependencies as an error).
9. If a web application type was detected, an **embedded server** (Tomcat by default) is created and started, bound to `server.port` .
10. **`ApplicationRunner`** / **`CommandLineRunner`** beans, if any, run after context startup completes.
11. The app is now up and serving requests.

3.3 How a Bean Actually Gets Created (IoC Container Internals)

- Every `@Component` / `@Bean` becomes a `BeanDefinition` — essentially a recipe (class, constructor args, scope, lazy/eager, etc.), stored in a `BeanFactory` (specifically a `DefaultListableBeanFactory`).
- For singleton beans (the default), Spring **eagerly instantiates all of them at startup** (unless marked `@Lazy`), in dependency order — a bean that depends on another is created *after* its dependency.
- Instantiation uses reflection to call the constructor Spring picks (if there's exactly one constructor, Spring uses it automatically for injection — no `@Autowired` annotation needed on it since Spring Framework 4.3+).
- After construction, Spring processes any `@PostConstruct` methods, then the bean is fully initialized and placed into the **singleton cache** (a `ConcurrentHashMap` of bean name → instance) inside the `ApplicationContext` .
- Every future `@Autowired` /constructor-injection request for that type is served from this cache — that's why singleton beans are shared, mutable-state-sensitive objects: there's exactly one instance for the whole application (per `ApplicationContext`).

3.4 How a Request Flows Through a `@RestController`

1. The embedded Tomcat receives the raw HTTP request and hands it to Spring's front controller, **`DispatcherServlet`** — the single entry point for all Spring MVC requests.
2. `DispatcherServlet` consults a `HandlerMapping` to find which controller method matches the request's URL + HTTP verb (built from your `@GetMapping` / `@PostMapping` / `@RequestMapping` annotations).
3. `HandlerAdapter` invokes that method, resolving each parameter — `@RequestBody` triggers Jackson to deserialize the JSON body into your Java object; `@PathVariable` / `@RequestParam` extract values from the URL.
4. Your controller method runs (typically delegating straight to a `@Service`).
5. The method's return value is passed to a `HttpMessageConverter` (Jackson, by default) which serializes it back to JSON for the response body — this automatic serialization is what `@RestController` (`@Controller` + `@ResponseBody`) gives you over plain `@Controller`, which would instead try to resolve a view template.
6. The response is written back through Tomcat to the client.

3.5 The Executable "Fat JAR"

`spring-boot-maven-plugin` repackages your compiled classes plus *all* your dependencies' `.class` /resource files into one self-contained JAR, with a custom `Launcher` class as the actual `Main-Class` (not your own `main()` directly) — this launcher sets up a special classloader capable of loading nested JARs-within-a-JAR (`BOOT-INF/lib/*.jar`), which the standard JVM classloader can't do on its own. That's why `java -jar app.jar` just works without needing a classpath flag listing every dependency.

PART 4: REAL-WORLD EXAMPLES ([Real-World])

1. **Microservices at scale**: companies running dozens of Spring Boot microservices rely heavily on `spring-boot-starter-parent`'s version alignment to avoid "dependency hell" across services maintained by different teams — a shared internal parent POM often extends this further with company-wide standard versions.
2. **Zero-downtime deploys**: the embedded-server model (no separate Tomcat install to manage) is what makes Spring Boot apps trivially containerizable — a Dockerfile is often just `COPY app.jar` + `ENTRYPOINT java -jar app.jar`, which is a big part of why Spring Boot became the default choice for Java in Kubernetes environments.
3. **Feature flags / environment-specific behavior**: `application-{profile}.properties` combined with `@Profile("prod")` beans lets teams run the exact same JAR in staging and production, swapping out things like mock payment gateways for real ones purely via `spring.profiles.active`, with no code changes.
4. **Auto-configuration debugging in production incidents**: a service unexpectedly picks up an embedded H2 database instead of the intended production Postgres because a test-scoped dependency leaked into the main classpath — a real, recurring class of bug that's understood by knowing `DataSourceAutoConfiguration`'s `@ConditionalOnClass` behavior (it activates for *whichever* JDBC driver it finds on the classpath).
5. **Circular dependency crashes at startup**: two `@Service` beans injecting each other via constructor injection cause Spring to fail fast at startup with a `BeanCurrentlyInCreationException`, rather than silently deadlocking — this "fail fast, not fail late" behavior is a deliberate design choice interviewers like to probe, since it's a real bug class caught immediately in CI rather than in production.
6. **Externalized secrets**: production systems avoid hardcoding credentials in `application.properties` by using `${DB_PASSWORD}` placeholders resolved from environment variables or a secrets manager (AWS Secrets Manager, Vault) injected at container startup — never committed to source control.

PART 5: INTERVIEW QUESTIONS ([Q&A] 36 Q&A)

1. What's the fundamental difference between Spring and Spring Boot? Spring is the underlying framework (DI/IoC container plus modules like Spring MVC, Spring Data); Spring Boot is an opinionated layer on top that auto-configures Spring for you and bundles an embedded server, eliminating most manual setup.

2. What problem does Dependency Injection solve? It removes tight coupling between a class and its dependencies' concrete implementations, making code easier to test (swap in mocks) and easier to change (swap implementations without touching the dependent class).

3. What's the difference between IoC and DI? IoC is the broader principle — control over object creation/wiring is inverted from your code to a container. DI is the specific pattern Spring uses to implement IoC (dependencies are injected rather than self-constructed).

4. Why is constructor injection generally preferred over field injection? It allows `final` fields (immutability), makes dependencies explicit and visible in the constructor signature, fails fast at object-construction time if a dependency is missing, and allows plain `new` instantiation in unit tests without needing a Spring context or reflection-based mocking.

5. What is a Spring bean? Any object whose lifecycle (creation, configuration, wiring, destruction) is managed by the Spring IoC container, typically declared via `@Component`-family annotations or explicit `@Bean` methods.

6. What's the difference between `@Component` and `@Bean`? `@Component` is a class-level annotation picked up by component scanning — for classes you own. `@Bean` is a method-level annotation inside a `@Configuration` class, used when you need custom construction logic or don't own the class's source (e.g., configuring a third-party client).

7. What's the default bean scope, and name two others? Singleton (one instance per container) is the default; others include `prototype` (new instance per request for the bean) and web-aware scopes like `request` / `session`.

8. What does `@Autowired` do? Tells Spring to resolve and inject a matching bean for that field/constructor/setter parameter from the `ApplicationContext`. Since Spring 4.3+, it's optional on a single constructor — Spring infers injection automatically.

9. What happens if Spring finds two beans matching the same type for injection? It throws a `NoUniqueBeanDefinitionException` unless disambiguated — via `@Primary` on one candidate, or `@Qualifier("beanName")` at the injection point.

10. What is `@Configuration` and how does it relate to `@Bean`? Marks a class as a source of bean definitions; `@Bean`-annotated methods inside it are called by Spring (not directly by your code) to produce beans, which are then cached and managed like any other bean.

11. What is the role of `pom.xml`'s `<parent>` section with `spring-boot-starter-parent`? It centralizes and pins dependency versions across the whole Spring Boot ecosystem so individual `<dependency>` entries don't need explicit `<version>` tags — avoiding version-mismatch issues between, say, Spring MVC and Jackson.

12. What's a Maven "starter" dependency? A curated, no-version-needed bundle of related dependencies for a specific purpose — e.g., `spring-boot-starter-web` brings in Spring MVC, embedded Tomcat, and Jackson together, all mutually compatible versions.

13. What Maven command builds a Spring Boot project into an executable JAR? `mvn clean package` (produces the JAR in `target/`), or `mvn spring-boot:run` to build and run directly without producing a distributable artifact.

14. What is `application.properties` for, and what's the alternative format? Centralized, externalized configuration (server port, DB connection, logging levels, custom app properties), loaded automatically from `src/main/resources`. YAML (`application.yml`) is the common alternative, often preferred for nested/hierarchical config.

15. What are Spring profiles, and why use them? Named sets of configuration (`application-dev.properties`, `application-prod.properties`) activated via `spring.profiles.active`, letting the same codebase/JAR run with different configuration (or different bean implementations, via `@Profile`) per environment.

16. What's the standard Spring Boot project package structure, and why does it matter? Typically layered as controller → service → repository → model/entity, all under one base package that the `@SpringBootApplication` class sits at the root of — because `@ComponentScan` only scans that package and below by default.

17. What does `@SpringBootApplication` actually consist of? It's shorthand for `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`.

18. What is auto-configuration, mechanically? Spring Boot scans a bundled list of candidate `@Configuration` classes (from `AutoConfiguration.imports`), each guarded by conditional annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`), and only activates the ones whose conditions are satisfied by your classpath/configuration/existing beans.

19. How would you override an auto-configured bean with your own? Define your own `@Bean` of the same type in a `@Configuration` class — most auto-configuration is guarded by `@ConditionalOnMissingBean`, so it backs off automatically once you supply your own.

20. What is `DispatcherServlet` and what role does it play? Spring MVC's front controller — the single entry point for every incoming HTTP request, responsible for routing to the right controller method, invoking it, and converting the result into an HTTP response.

21. What's the difference between `@Controller` and `@RestController`? `@RestController` = `@Controller` + `@ResponseBody` — method return values are serialized directly into the HTTP response body (typically JSON), instead of being resolved as a view template name.

22. What is an embedded server, and why does Spring Boot use one? A servlet container (Tomcat, by default) bundled directly inside the application rather than installed separately — it's what lets a Spring Boot app run as a plain, self-contained `java -jar`, which is central to how easily it containerizes.

23. How does Spring Boot package an executable "fat JAR," and why can't the JVM run it like a normal JAR? It bundles the app's classes plus all dependency JARs inside one JAR (nested under `BOOT-INF/lib/`) with a custom launcher as the entry point, because the standard JVM classloader can't natively load JARs nested inside another JAR — Spring Boot's launcher provides a special classloader that can.

24. What is `@Value` used for? Injecting a single externalized property value directly into a field: `@Value("${server.port}") private int port;`.

25. What's the difference between `@Value` and `@ConfigurationProperties`? `@Value` injects one property at a time; `@ConfigurationProperties` binds a whole prefixed group of properties onto a type-safe class/record in one go — generally preferred for related, multi-property configuration blocks.

26. What is a circular dependency, and how does Spring Boot handle it? Two beans depending on each other (directly or transitively). With constructor injection, Spring fails fast at startup with a `BeanCurrentlyInCreationException` rather than allowing it — forcing you to break the cycle (often via redesign or, as a last resort, setter injection with `@Lazy`).

27. What is `@PostConstruct` used for? Marking a method to run automatically right after a bean's dependencies are injected and construction completes — common for initialization logic that needs the injected dependencies already set.

28. How would you expose a health-check endpoint in a Spring Boot app? Add `spring-boot-starter-actuator`, which exposes `/actuator/health` (and other operational endpoints like `/actuator/metrics`) out of the box, configurable via properties.

29. What's the difference between `@RequestParam` and `@PathVariable`? `@RequestParam` binds a query-string parameter (`?name=value`); `@PathVariable` binds a segment of the URL path itself (`/employees/{id}`).

30. How does Spring Boot decide it should be a "web application" and start an embedded server? By inspecting the classpath during `SpringApplication.run()` — presence of `spring-webmvc` (Servlet stack) or `spring-webflux` (Reactive stack) determines the application type and whether/which embedded server gets started.

31. What is `CommandLineRunner` / `ApplicationRunner` used for? Beans implementing these interfaces have their `run()` method automatically invoked once, right after the application context has fully started — commonly used for startup tasks like seeding data.

32. Why might a bean unexpectedly not be found/injected at runtime? Common causes: it's outside the `@ComponentScan` base package; a conditional annotation on an auto-configuration didn't pass; the class is missing its stereotype annotation; or there's an ambiguous multi-candidate injection that was never disambiguated.

33. What does `spring.jpa.hibernate.ddl-auto=update` do, and why is it dangerous in production? Tells Hibernate to automatically alter the DB schema to match your entities at startup. Convenient in development, but risky in production since it can silently make unintended schema changes — teams typically use `validate` or migration tools (Flyway/Liquibase) instead.

34. How would you test a `@Service` class without starting the whole Spring context? Instantiate it directly with `new` and pass mocked dependencies (e.g., Mockito mocks) via the constructor — this is exactly why constructor injection is preferred, since it makes plain unit testing possible without `@SpringBootTest`.

35. What's the difference between `@SpringBootTest` and a plain unit test with Mockito? `@SpringBootTest` boots the (or a slice of the) actual Spring `ApplicationContext` — slower, but tests real wiring/integration behavior. A plain Mockito-based test instantiates the class under test directly with mocked collaborators — fast, isolated, and doesn't need Spring at all.

36. How does Spring Boot's precedence order work when the same property is set in multiple places (e.g., `application.properties` vs. an environment variable vs. a command-line argument)? Command-line arguments generally take highest precedence, then environment variables, then profile-specific properties files, then the base `application.properties` — letting ops override any file-based default without touching the packaged JAR.

PART 6: CODING PROBLEMS ([Coding Problems])

Problem 1 — Add a New REST Endpoint

Task: extend the `EmployeeController` from Part 2 with an endpoint that returns all employees in a given department.

```
        // Repository already has: findByDepartment(String department)

        @GetMapping("/department/{dept}")
        public List<Employee> byDepartment(@PathVariable String dept) {
            return repository.findByDepartment(dept);   // or via service, keeping layering consistent
        }
```

Problem 2 — Fix the Field-Injection Anti-Pattern

Given this code, refactor it to use constructor injection and explain why it's better for testing.

```
        // BEFORE
        @Service
        public class OrderService {
            @Autowired
            private PaymentGateway gateway;
        }
```

```
        // AFTER
        @Service
        public class OrderService {
            private final PaymentGateway gateway;

            public OrderService(PaymentGateway gateway) {
                this.gateway = gateway;
            }
        }
```

Now `new OrderService(mockGateway)` works in a plain unit test with no Spring context, and `gateway` can be `final`, guaranteeing it's never reassigned after construction.

Problem 3 — Resolve an Ambiguous Bean Injection

Given two implementations of `NotificationSender` (`EmailSender`, `SmsSender`), fix the compile-time-passing-but-runtime-failing injection below.

```
        public interface NotificationSender { void send(String msg); }

        @Component
        public class EmailSender implements NotificationSender { ... }

        @Component
        public class SmsSender implements NotificationSender { ... }

        @Service
        public class AlertService {
            private final NotificationSender sender;   // ambiguous: 2 candidates!
            public AlertService(NotificationSender sender) { this.sender = sender; }
        }
```

Fix, option A — mark a default:

```
        @Component
        @Primary
        public class EmailSender implements NotificationSender { ... }
```

Fix, option B — disambiguate at the injection point:

```
        public AlertService(@Qualifier("smsSender") NotificationSender sender) {
            this.sender = sender;
        }
```

Problem 4 — Externalize a Hardcoded Value

Task: move a hardcoded API key out of the code and into configuration, with a safe default for local dev.

```
        // BEFORE
        private static final String API_KEY = "sk-hardcoded-12345";
```

```
# application.properties
app.payment.api-key=${PAYMENT_API_KEY:sk-local-dev-only}
```

```
        // AFTER
        @Value("${app.payment.api-key}")
        private String apiKey;
```

Problem 5 — Diagnose a Startup Failure

Given this stack trace fragment:

```
BeanCurrentlyInCreationException: Error creating bean with name 'orderService':
Requested bean is currently in creation: Is there an unresolvable circular reference?
```

Task: explain the likely cause and two ways to fix it. Likely cause: `OrderService` and another bean (say, `InventoryService`) inject each other via constructor, forming a cycle Spring can't resolve at construction time. Fixes: (1) redesign to remove the cycle — e.g., extract shared logic into a third bean both depend on; (2) as a last resort, break the cycle with setter injection plus `@Lazy` on one side, so one bean's dependency is resolved lazily after both exist.

Problem 6 — Write a `@ConfigurationProperties` Class

Task: bind these properties to a type-safe class instead of scattered `@Value` fields.

```
app.mail.host=smtp.example.com
app.mail.port=587
app.mail.from=noreply@example.com
```

```
        @ConfigurationProperties(prefix = "app.mail")
        public record MailProperties(String host, int port, String from) {}
```

```
        @Configuration
        @EnableConfigurationProperties(MailProperties.class)
        public class MailConfig {}
```

PART 7: AI-STYLE INTERVIEW QUESTIONS ([AI])

1. "A junior engineer asks why they can't just use `new EmployeeService()` everywhere instead of letting Spring inject it. How do you explain the trade-off?" Expected reasoning: `new` works fine for simple, dependency-free objects, but `EmployeeService` itself depends on a repository, which depends on a datasource — manually wiring that whole graph by hand for every object is exactly what Spring's IoC container automates. The real payoff shows up at the scale of dozens of interdependent beans, not in a single toy example — and it's what makes swapping implementations (e.g., mocking in tests) trivial.

2. "Your team wants to shave startup time. Auto-configuration seems to run a lot of unnecessary checks. Would you disable it?" Expected reasoning: don't disable it wholesale — that throws away most of Spring Boot's value. Instead, profile actual startup time (`--debug` flag shows the auto-configuration report of what was applied/skipped and why), exclude specific unneeded auto-configurations via `@SpringBootApplication(exclude = ...)` if a particular one is genuinely unused and costly, and consider lazy initialization (`spring.main.lazy-initialization=true`) for non-critical-path beans rather than turning off the whole mechanism.

3. "You inherit a service with heavy field injection and no tests. How do you approach adding tests without a large rewrite?" Expected reasoning: start by refactoring just the class under test to constructor injection (a low-risk, mechanical change), which immediately unlocks plain Mockito-based unit tests without booting a Spring context; reserve `@SpringBootTest` for a smaller number of true integration tests, since booting the full context for every test class doesn't scale as the codebase grows.

4. "A teammate proposes putting all configuration in `application.properties`, including secrets, and committing it to the repo for 'convenience.' What's your response?" Expected reasoning: push back — secrets in source control are a real, common security incident source; use placeholder syntax (`${DB_PASSWORD}`) resolved from environment variables or a secrets manager at deploy time instead, keeping `application.properties` itself safe to commit.

5. "Production is throwing `NoUniqueBeanDefinitionException` intermittently, only after a new library was added as a dependency. How would you investigate?" Expected reasoning: "intermittently" is the key detail — Spring bean resolution failures aren't actually non-deterministic; more likely the new library auto-configures its own bean of a type you already had, and which candidate "wins" depends on classpath/bean-registration order, which *can* vary. Check whether the new dependency is a Spring Boot starter with its own auto-configuration for a type you already define — the fix is usually `@Primary` or `@Qualifier`, or excluding the library's auto-configuration.

6. "How would you decide whether a new piece of shared logic belongs in a `@Service` bean or a plain utility class (static methods, not Spring-managed)?" Expected reasoning: if it's stateless, pure logic with no dependencies to inject (e.g., date formatting, string utilities), a plain static utility class is simpler and doesn't need DI. If it depends on other beans (a repository, another service, configuration) or needs to be mockable/swappable in tests, it belongs in a Spring-managed bean.

7. "An AI coding assistant generated a `@Bean` method for a class you already annotated with `@Component`. What's wrong, and why might it cause a subtle bug rather than an obvious error?" Expected reasoning: that class now has two separate bean-registration paths — component scanning and the explicit `@Bean` method — which can register it twice under different bean names, or throw `ConflictingBeanDefinitionException` depending on configuration, and if scopes/config differ between the two registrations, different parts of the app could end up wired to *different* instances without any obvious error, which is a much harder bug to trace than a hard failure.

8. "Your team is deciding between YAML and `.properties` for configuration going forward. What would you actually weigh, beyond personal preference?" Expected reasoning: YAML expresses nested/hierarchical config (like profile-specific blocks in one file via `---` document separators) more readably; `.properties` is flatter but has no indentation-sensitivity bugs and is arguably safer for large teams less familiar with YAML's whitespace rules. Neither is functionally superior — the real trade-off is team familiarity and how nested the actual configuration needs to be.

PART 8: MOCK INTERVIEW ([Mock Interview])

A realistic back-and-forth transcript.

Interviewer: Let's start at a high level — what's the actual difference between Spring and Spring Boot? I find a lot of candidates say "Spring Boot is easier" without explaining why.

You: Spring is the underlying framework — the DI container, Spring MVC, Spring Data, all of it. It's powerful but you have to wire most of it yourself: XML or Java config, manually declaring beans, deploying a WAR to an external Tomcat. Spring Boot sits on top of that. It doesn't replace any of it — it auto-configures Spring based on what's on your classpath, bundles an embedded server, and gives you curated starter dependencies so you're not manually aligning versions. So the "easier" part specifically comes from auto-configuration and convention over configuration, not from Spring Boot being a different framework underneath.

Interviewer: Good, that's the distinction I wanted. Let's go a level deeper — walk me through what actually happens when `SpringApplication.run()` executes.

You: At a high level: Spring Boot figures out what kind of application this is — servlet web, reactive, or none — based on what's on the classpath. It builds the right `ApplicationContext` type, merges configuration from properties files, environment variables, and command-line args into an `Environment`. Then component scanning runs and registers bean definitions for everything under the base package. Auto-configuration then evaluates a big list of candidate configuration classes, each gated by conditions like "is this class on the classpath" or "has the user already defined this bean" — only the ones that pass get applied. Then the container actually instantiates all the singleton beans, resolving the dependency graph. If it's a web app, an embedded Tomcat starts up bound to the configured port, and then the app is live.

Interviewer: Nice, that's thorough. Quick follow-up: what stops auto-configuration from just overriding a bean I define myself?

You: Most auto-configuration classes are annotated with `@ConditionalOnMissingBean`. So if I've already registered my own bean of that type — say, my own `DataSource` — the auto-configured one backs off and doesn't get created at all. That's the mechanism that makes it safe to override defaults without conflicts.

Interviewer: Let's talk dependency injection specifically. Why do you prefer constructor injection over field injection with `@Autowired`?

You: A few reasons. Constructor injection lets the dependency field be `final`, so it's immutable after construction and can't accidentally get reassigned later. It makes the dependencies visible right in the constructor signature instead of hidden as private fields scattered through the class. It fails fast — if a required dependency can't be resolved, the object literally can't be constructed. And practically, it's what makes plain unit testing possible: I can call `new OrderService(mockGateway)` directly in a test with no Spring context involved at all, versus needing reflection tricks to inject into private fields.

Interviewer: Suppose I have two implementations of the same interface and I try to constructor-inject that interface type. What happens?

You: Spring can't decide which one to give you, so it throws a `NoUniqueBeanDefinitionException` at startup. To fix it, I'd either mark one implementation `@Primary` as the default, or use `@Qualifier` with the specific bean name at the injection point if I need to choose explicitly at each call site.

Interviewer: Last question — and this one's a judgment call, not a definition. Your team wants to disable most of Spring Boot's auto-configuration to speed up startup time. What do you actually do?

You: I wouldn't disable it wholesale — that gives up most of what makes Spring Boot productive in the first place, and it's a big surface area to now maintain manually. I'd first actually measure where the time is going — Spring Boot can print an auto-configuration report showing exactly what got applied and what got skipped and why. If specific auto-configurations are genuinely unused and expensive, I'd exclude just those explicitly. And if the real issue is a lot of beans getting eagerly created at startup that aren't needed on the critical path, lazy initialization is a much more targeted fix than turning auto-configuration off entirely.

Interviewer: That's a strong answer — targeted over wholesale is exactly the instinct I was looking for. Thanks, that's everything I had.

End of guide.